LinkedList

LinkedList

- A linked list is a non-primitive linear data structure, in which the elements are not stored at contiguous memory locations.

- It is a collection of items and each item is represented by a node.

Cont…

- Each node will contain at least two field.
- First field will contain information of element is called data and this field also called data field.
- Second field is called pointer field and it will contain the address of next node.

Cont…

- Here, we will have a START pointer that will contain the address of first node of a linkedlist.

Types of LinkedList

- There are different types of linkedlist.
  - Single LinkedList (Linear LinkedList)
  - Circular Single LinkedList
  - Doubly LinkedList
  - Circular Doubly LinkedList

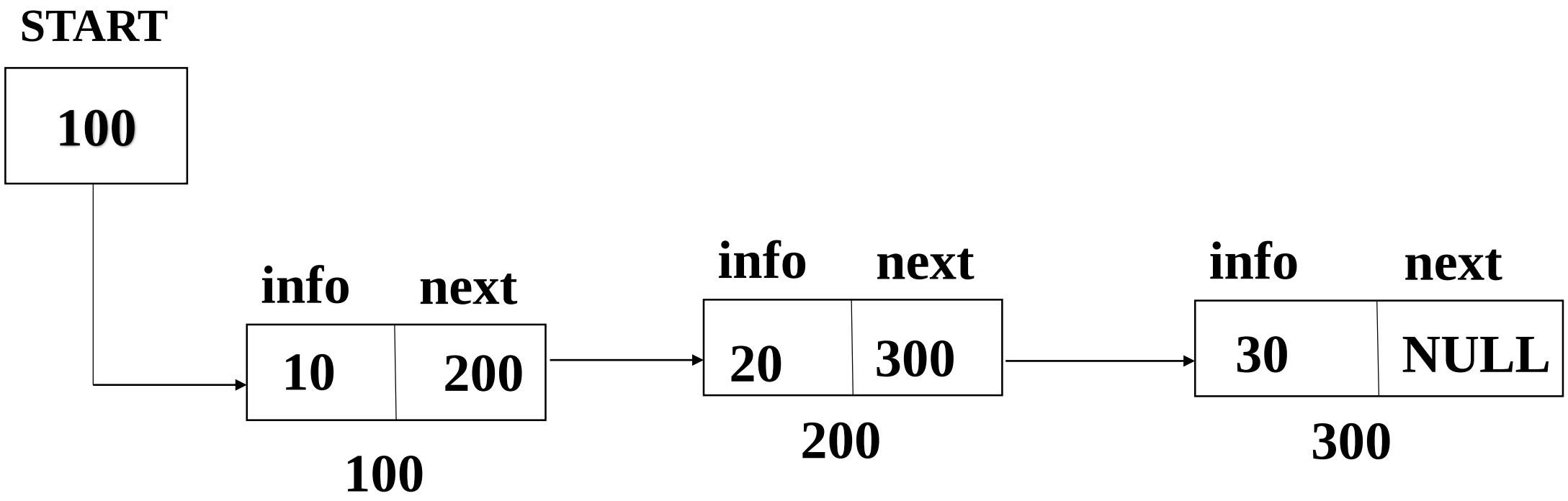
1. Single LinkedList (Linear LinkedList)

- In it, each node will contain 2 fields.

- First field will contain the information of the element is called data field and second field contain the address of next node is called pointer field.

- Pointer field of last node will contain NULL value.

Example of Single LinkedList



Disadvantage of Single Linked List

- In it, we can not access previous node from the current node.

- This problem can be removed in doubly LinkedList.

Operations on Linked List

- Following are important operations on Linked List

- Inserting Element in a linked list
- Traversing/Processing/Printing a Linked List
- Deleting Element from a linked list
- Count number of elements in a linked list
- Search an element in a linked list

Operations on Linked List

- Following are important operations on Linked List

- Insert
- Traverse
- Delete
- Count
- Search

Traverse

- **Traversal of Singly Linked List**
- Traversal in a linked list means visiting each node and performing operations like printing or processing data.

Traverse

- Step-by-step approach:

1. Initialize a pointer (current) to the head of the list.
2. Loop through the list using a while loop until current becomes NULL.
3. Process each node (e.g., print its data).
4. Move to the next node by updating current = current->next.

Algorithm 5.1:

(Traversing a Linked List) Let LIST be a linked list in memory. This algorithm traverses LIST, applying an operation PROCESS to each element of LIST. The variable PTR points to the node currently being processed.

1. Set PTR := START. [Initializes pointer PTR.]
2. Repeat Steps 3 and 4 while PTR ≠ NULL.
3. Apply PROCESS to INFO[PTR].
4. Set PTR := LINK[PTR]. [PTR now points to the next node.]

   [End of Step 2 loop.]
5. Exit.

Search

- Searching in Singly Linked List
- Searching in a Singly Linked List refers to the process of looking for a specific element or value within the elements of the linked list.

Search

- **Step-by-step approach:**

1. Start from the head of the linked list.
2. Check each node’s data:
3. If it matches the target value, return true (element found).
4. Otherwise, move to the next node.
5. Repeat until the end (NULL) is reached.
6. If no match is found, return false.

Algorithm 5.2

SEARCH(INFO, LINK, START, ITEM, LOC)
LIST is a linked list in memory. This algorithm finds the location LOC of the node where ITEM first appears in LIST, or sets LOC = NULL.

1. Set PTR := START.
2. Repeat Step 3 while PTR ≠ NULL:
3. If ITEM = INFO[PTR], then:
   Set LOC := PTR, and Exit.
   Else:
   Set PTR := LINK[PTR]. [PTR now points to the next node.]
   [End of If structure.]
   [End of Step 2 loop.]
4. [Search is unsuccessful.] Set LOC := NULL.
5. Exit.

Count/Length

- Length of Singly Linked List
- Finding the length of a Singly Linked List means counting the total number of nodes.

Count/Length

- **Step-by-step approach:**

- Initialize a counter (length = 0).
- Start from the head, assign it to current.
- Traverse the list:
- Increment length for each node.
- Move to the next node (current = current->next).
- Return the final length when current becomes NULL.

Example 5.7

The following procedure finds the number NUM of elements in a linked list.

Procedure: COUNT(INFO, LINK, START, NUM)

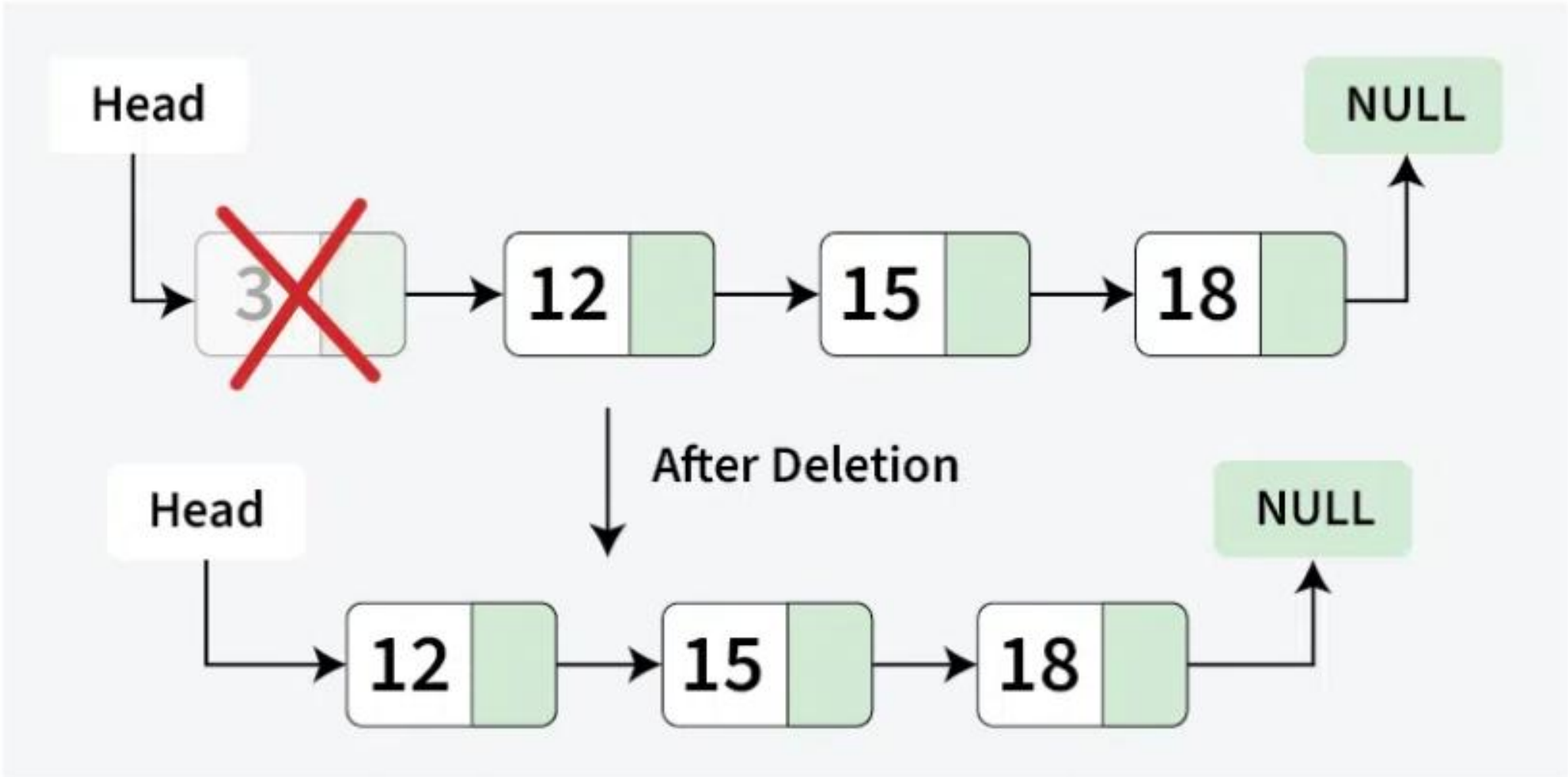
1. Set NUM : = 0. [Initializes counter.]
2. Set PTR : = START. [Initializes pointer.]
3. Repeat Steps 4 and 5 while PTR ≠ NULL.
4. Set NUM : = NUM + 1. [Increases NUM by 1.]
5. Set PTR : = LINK[PTR]. [Updates pointer.]
   [End of Step 3 loop.]
6. Return.

Delete

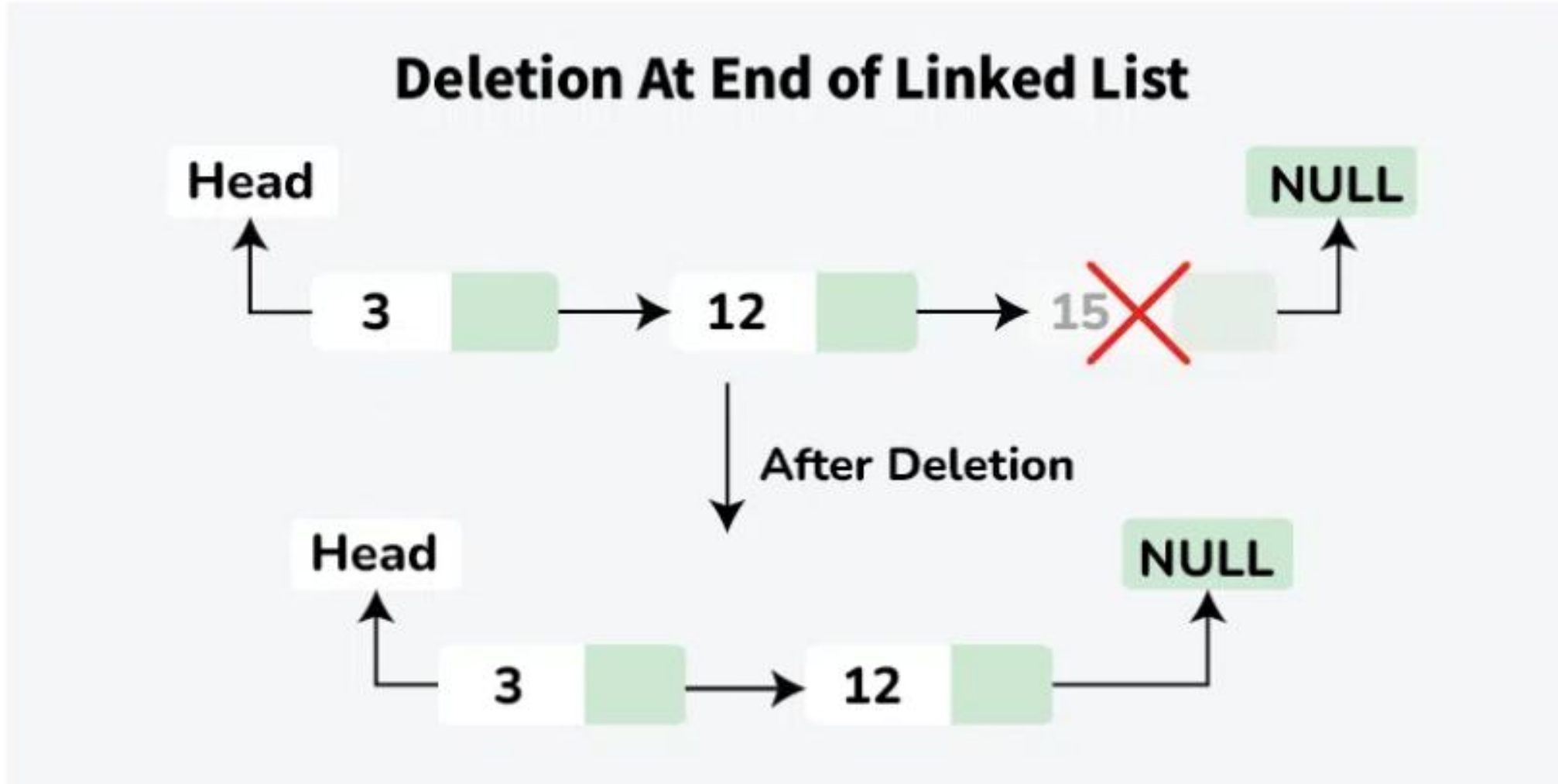
- **Deletion in Singly Linked List**
- Deleting a node in a Linked List is an important operation and can be done in three main ways:
- removing the first node,
- removing a node in the middle,
- or removing the last node.

Delete

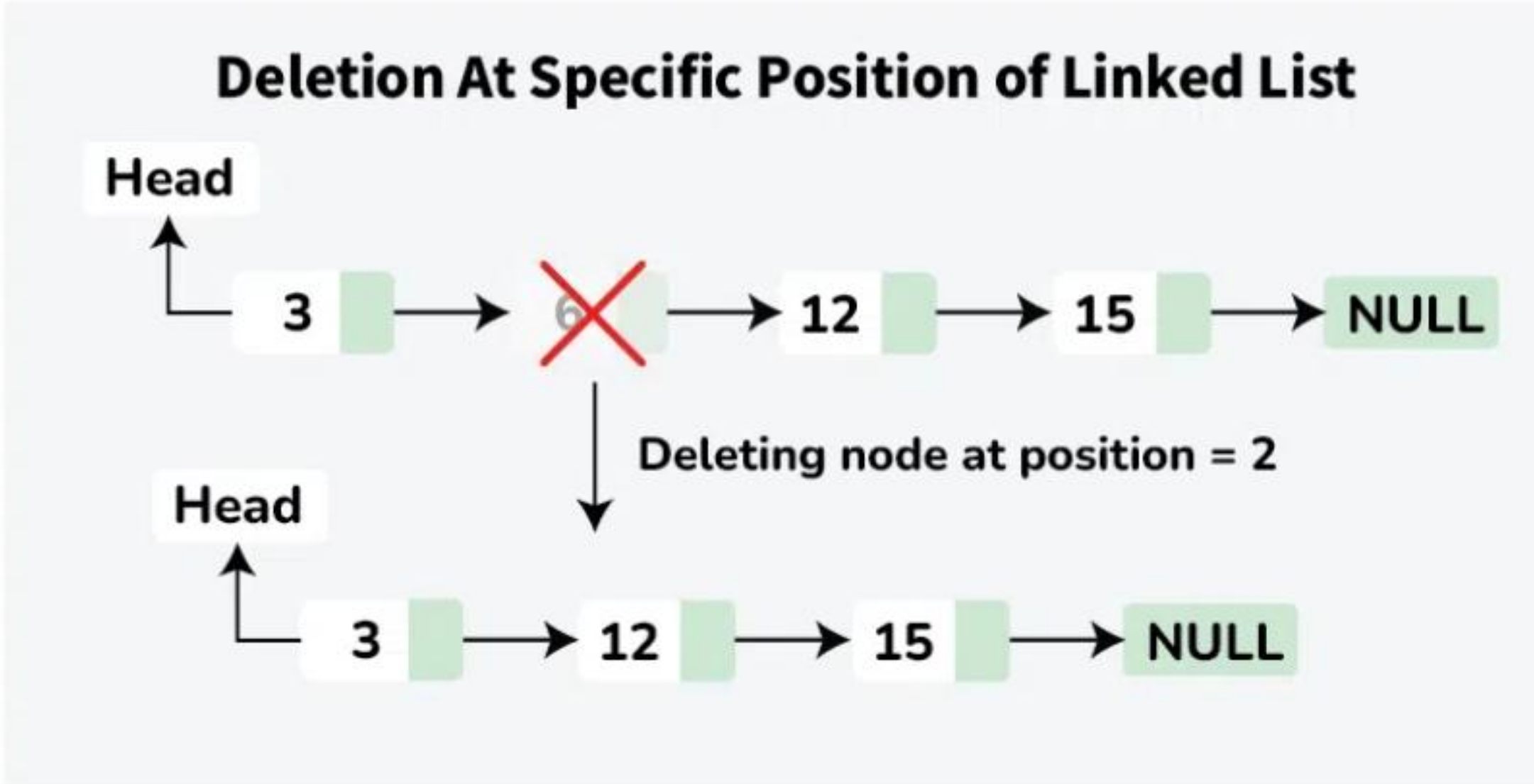
- There are different scenarios for deletion:
- **Deletion at the Beginning of Singly Linked List:**
- **Deletion at the End of Singly Linked List:**
- **Deletion at a Specific Position of Singly Linked List:**



Deletion at beginning in a Linked List



Deletion at the end of linked list



Delete a Linked List node at a given position

Algorithm is to delete a node containing a specified key

Purpose:

The purpose of this algorithm is to delete a node containing a specified key value from a singly linked list.

The algorithm traverses the list to locate the node with the target value and removes it while maintaining the integrity of the linked list.

Problem Statement:

Given a singly linked list and a key value, delete the node that contains the specified key.

Update the list and ensure all pointers are correctly adjusted.

Algorithm is to delete a node containing a specified key

Input:

A pointer to the head of the linked list.

An integer key that represents the value of the node to be deleted.

Output:

The modified linked list after deleting the node with the given key.

If the key is not found, the list remains unchanged.

Constraints:

The list may be empty.

The key may be located anywhere in the list.

The node to be deleted may be the first, middle, or last node.

Algorithm is to delete a node containing a specified key

Algorithm:

Step 1: Start

Define a function deleteNode(head, key) that takes the head of the list and the key as arguments.

Step 2: Check if List is Empty

If head == null, print "List is empty" and return head.

Step 3: Check if Head Node Holds the Key

Check if the head node contains the key:

If head.data == key, update the head pointer to the next node using:

head = head.next

Return the updated list.

Algorithm is to delete a node containing a specified key

Step 4: Initialize Pointers

Create two pointers:

prev → Points to null initially.

current → Points to head.

Step 5: Traverse the List to Find the Key

Loop through the list until current != null:

If current.data == key:

Update the next pointer of the previous node:

prev.next = current.next

Break the loop and delete the node.

Else:

Move prev to current.

Move current to current.next.

Algorithm is to delete a node containing a specified key

Step 6: Check if Key was Found

If the key was not found (current == null), print "Key not found" and return head.

Step 7: Return Updated List

Return the modified list with the node deleted.

Algorithm is to delete a node containing a specified key

Pseudocode:

Function deleteNode(head, key):

If head == NULL:

Print "List is empty"

Return head

If head.data == key:

head = head.next

Return head

prev ← NULL

current ← head

While current != NULL:

If current.data == key:

prev.next = current.next

Break

prev ← current

current ← current.next

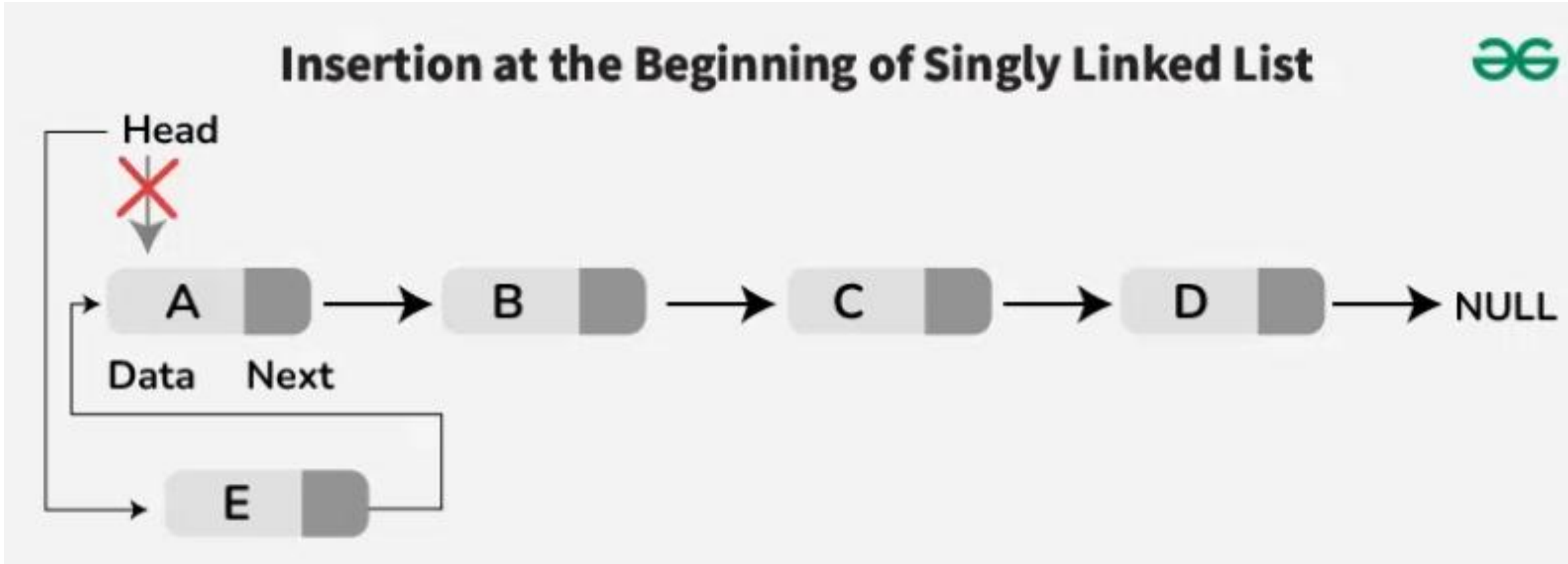
If current == NULL:

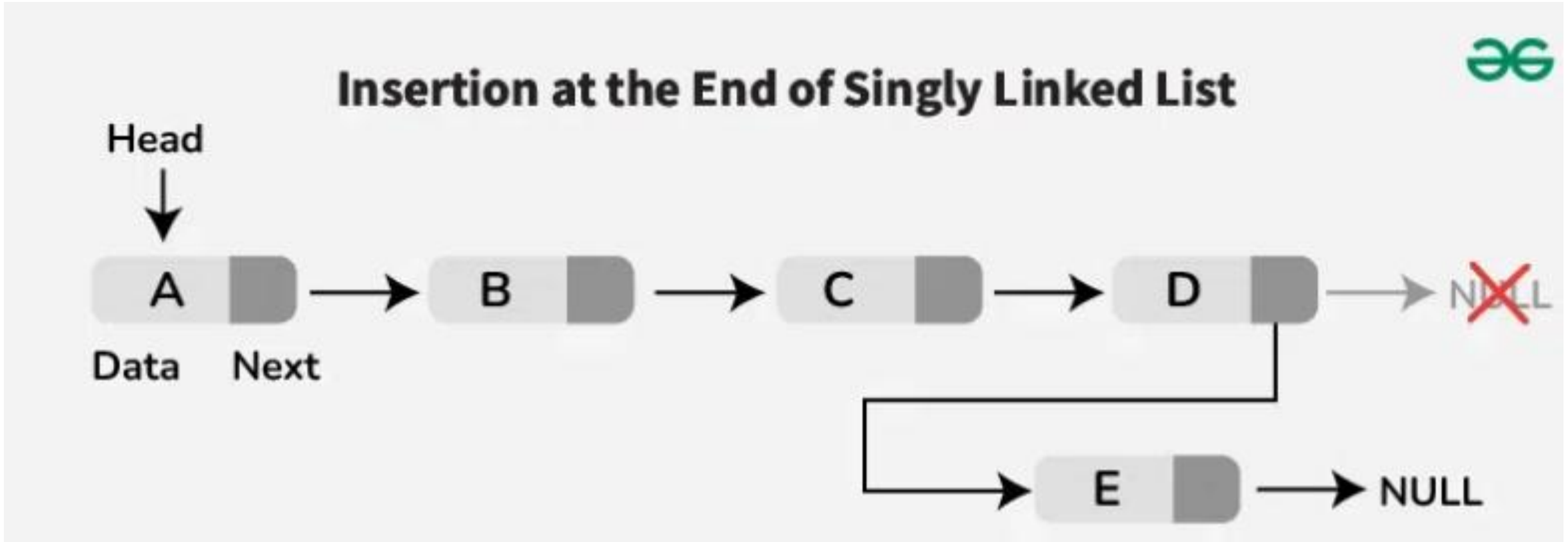
Print "Key not found"

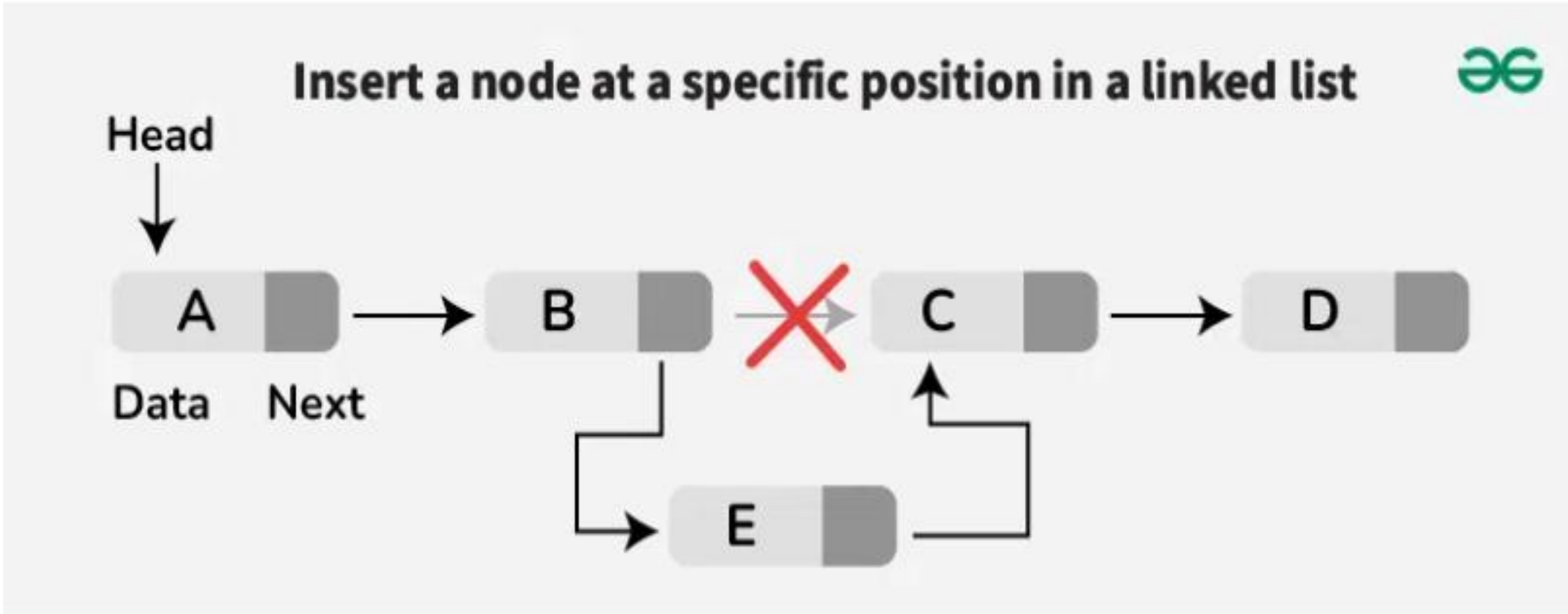
Return head

Insert

- Insertion is a fundamental operation in linked lists that involves adding a new node to the list. There are several scenarios for insertion:
- **Insertion at the Beginning of Singly Linked List:**
- **Insertion at the End of Singly Linked List:**
- **Insertion at a Specific Position of the Singly Linked List:**


Insertion at the Beginning of Singly Linked List
Head
A
B
C
D
NULL
Data
Next
E


Insertion at the End of Singly Linked List
Head
A
B
C
D
NULL
Data
Next
E
NULL


Insert a node at a specific position in a linked list
Head
A
Data
Next
B
C
D
E

Algorithm to Insert node

- The purpose of this algorithm is to **insert a node** with a given key value at a specified position in a singly linked list. The node can be inserted:
- At the **beginning**.
- At a **specific position**.
- At the **end** if the position exceeds the length of the list.

Algorithm to Insert node in a Linked List

Input:

- A pointer to the **head** of the linked list.
- An **integer key** representing the value to be inserted.
- An **integer position** representing the 0-based index where the node should be inserted.

Output:

- The modified linked list after inserting the new node at the specified position.
- If the position is invalid, the list remains unchanged, and an appropriate message is printed.

Algorithm to Insert node in a Linked List

Constraints:

The list may be empty.

If position == 0, the node is inserted at the beginning.

If position > length of list, the node is inserted at the end.

Position cannot be negative.

Algorithm to Insert node in a Linked List

Step 1: Start

Define a function insertAtPosition(head, key, position) that takes the head of the list, the key, and the position as arguments.

Step 2: Check for Invalid Position

If position < 0, print:

"Invalid position"

Return head.

Step 3: Create a New Node

Allocate memory for a new node.

Set:

newNode.data = key

newNode.next = NULL

Algorithm to Insert node in a Linked List

Step 5: Traverse the List to Find the Position

Create a pointer temp and initialize it to head.
Initialize a counter count = 0.
Traverse the list while temp != NULL and count < position - 1:
Move temp to temp.next.
Increment count.

Step 6: Check if Position is Out of Bounds

If temp == NULL and position > count + 1
print: "Position out of bounds, inserting at the end"
Traverse to the last node and insert the new node at the end.

Algorithm to Insert node in a Linked List

Step 7: Insert Node at Desired Position
If temp != NULL:
Update pointers to insert newNode:
newNode.next = temp.next
temp.next = newNode

Step 8: Return Updated List
Return the modified list after inserting the node.

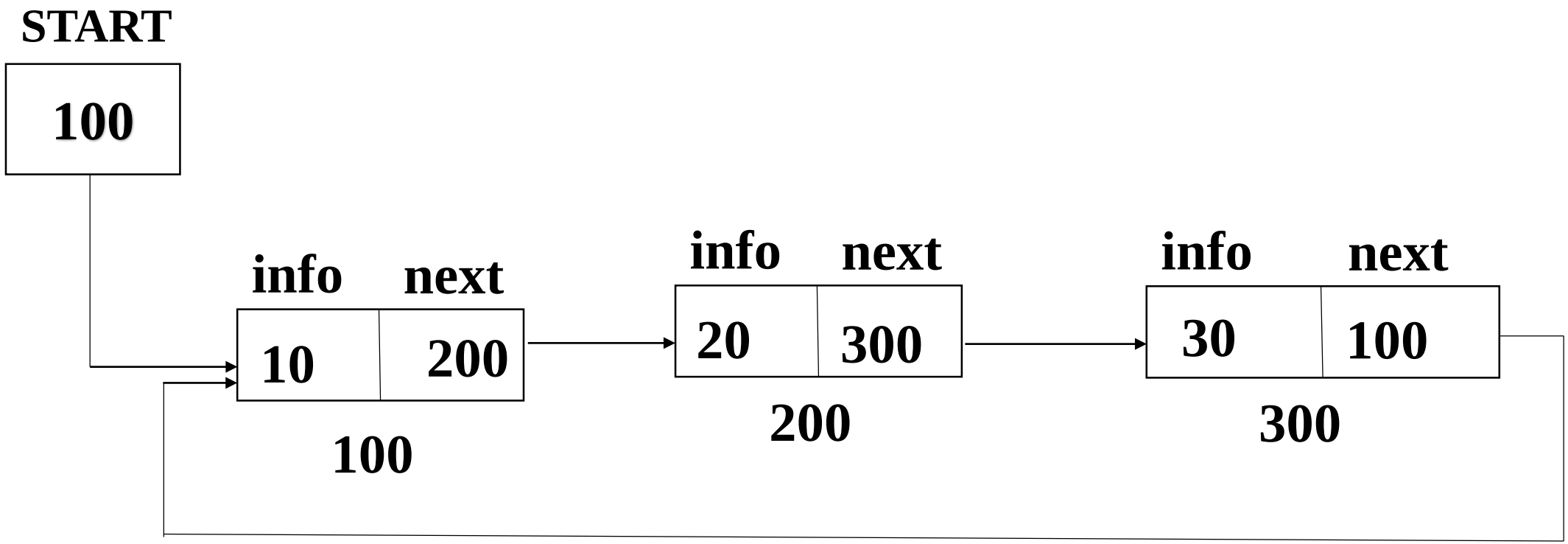
Pseudocode:

```
Function insertAtPosition(head, key, position):
    If position < 0:
        Print "Invalid position"
        Return head

    Create newNode
    newNode.data = key
    newNode.next = NULL

    If position == 0:
        newNode.next = head
        head = newNode
        Return head

    temp ← head
    count ← 0

    While temp != NULL AND count < position - 1:
        temp ← temp.next
        count ← count + 1

    If temp == NULL:
        Print “Inserting at the end"
        temp ← head
        While temp.next != NULL:
            temp ← temp.next
        temp.next = newNode
        Return head

    newNode.next = temp.next
    temp.next = newNode
    Return head
```

2. Circular Single LinkedList

- A single linkedlist can be made as circular single linkedlist in which pointer field of last node will contain the address of first node.

Example of Circular Single LinkedList



Advantage of Circular Single LinkedList

- In it, from last node we can move directly to the first node of circular single LinkedList because pointer field of last node will contain the address of first node.

Disadvantage of Circular Single Linked List

- In it, we can not access previous node from the current node.

- This problem can be removed in doubly LinkedList.

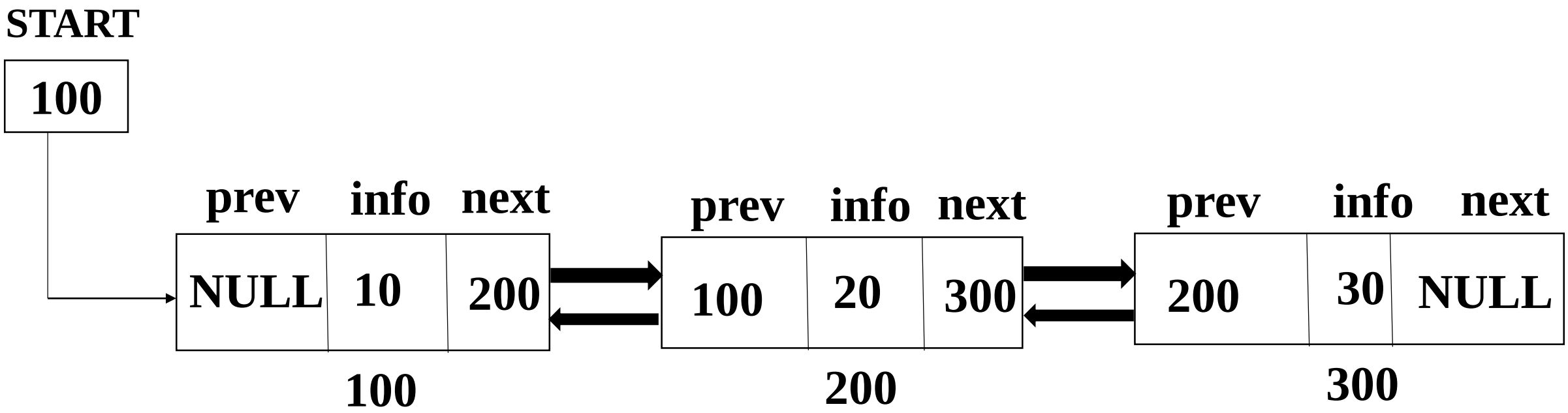
3. Doubly LinkedList

- It is also called Two-way linked list.
- Here each node will contain 3 fields. First field will contain the address of previous node. It is also called previous pointer.
- Second field will contain the information of element is called data field.

Cont…

- Third field will contain the address of next node is called next pointer.

Example of Doubly LinkedList



Advantage of Doubly LinkedList

- In it, we can we can move previous and next node from the current node because every node contain the address of previous and next node.

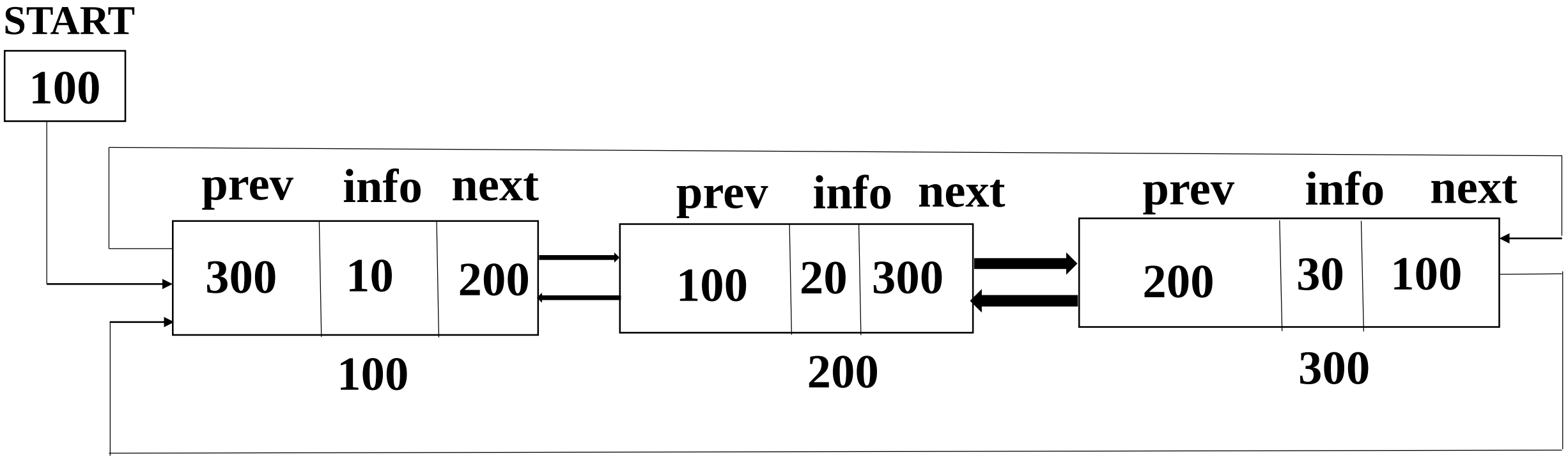
Disadvantage of Doubly LinkedList

- Doubly LinkedList require more memory than single linkedlist because every node of it contain 3 field. Two pointer and one data field.

4. Circular Doubly LinkedList

- A doubly linkedlist can be made as circular doubly linkedlist in which the previous pointer of first node will contain the address of last node and next pointer of last node will contain the address of first node.

Example of Circular Doubly LinkedList



Advantage of Circular Doubly LinkedList

- In it, from first node we can move directly to the last node because previous pointer of first node will contain the address of last node.

- In it, from last node we can move directly to the first node because next pointer of last node will contain the address of first node.

Advantage of LinkedList

- LinkedList is a dynamic data structure i.e they can increase or decrease a node during the execution of a program.

- The size is not fixed.

- Data can store non-continuous memory block.

Cont…

- Insertion and deletion of a node are easier and efficient.

Disadvantage of LinkedList

- **More Memory Required:** In the linked list there is an special field called pointer field which hold the address of next node so linked list requires extra space.

- Accessing to random data item is time consuming.

Syntax to create a node of LinkedList

```
struct structure name
{
    datatype variablename;
    struct structure name *pointervariablename;
};
```

Example:

```
struct node
{
    int info;
    struct node *next;
};
```

Cont…

Example:

```
struct node
{
    int info;
    struct node *next;
};
```